Algorithm Engineering: Course Syllabus & TOC

Part I: The Memory Hierarchy & Sorting

- IO Model and Basics
  - RAM vs. Two-Level Model (M, B, N)
  - I/O Analysis: Scanning, Binary Search vs. B-Trees
  - Virtual Memory Probabilistic Analysis
- External Sorting and Permuting
  - Sorting vs. Permuting Bottlenecks
  - MergeSort: Binary vs. Multi-way $(N/B \log_{M/B} N/M)$
  - Run Generation: Snow Plow $(2M)$
  - Disk Striping (D disks)
- Quicksort and Selection
  - In-Memory: 3-way, Dual Pivot, Bounded
  - Selection: Randomized Linear Time
  - External: Multi-way Quicksort & Oversampling

Part II: Streaming & Sets

- Random Sampling and Streams
  - Disk Sampling (Known N)
  - Reservoir Sampling (Unknown N, inductive proof)
- Intersection Algorithms
  - Merge-based vs. Binary Search-based
  - Galloping (Doubling) Search
  - Interpolation Search $(O(\log \log N))$

Part III: Advanced Data Structures

- Randomized Dictionaries
  - Skip Lists: Levels, Coin flips, I/O issues
  - Treaps: Rotations, Split/Merge, 3-sided Range Query
  - Random BST Average Depth Proof
- String Sorting and Tries
  - String Sorting Lower Bound $(d + n \log n)$
  - Multi-key Quicksort

- Radix Sorts (LSD/MSD)
- Tries: TST, Patricia, Front Coding

Part IV: Indexing & Hashing

- Full Text Indexing
  - Suffix Arrays & Binary Search
  - LCP Array: Kasai's Algorithm ($O(N)$)
  - Suffix Trees from SA+LCP
- Hashing Protocols
  - Universal Hashing
  - Power of Two Choices
  - Cuckoo Hashing: Graph theory & $O(1)$ lookup
- Filters and Perfect Hashing
  - Minimal Ordered Perfect Hashing (MOPHF)
  - Bloom Filters: False positive derivation
  - Spectral Bloom Filters

Part V: Compression

- Data compression
  - Entropy (H_0) & Kraft's Inequality
  - Huffman Coding (Canonical reconstruction)
  - Arithmetic Coding

IO Model and Basics

1. The Need for a New Model

The Von Neumann RAM Model

Historically, algorithm analysis has relied on the **Random Access Machine (RAM)** model (or Von Neumann model).

- **Assumptions:**
  - Infinite memory.
  - Uniform access time: Accessing any memory cell takes constant time $O(1)$.
  - Atomic operations: Arithmetic and logical operations take unit time.
- **Success:** It has been highly successful because it is simple and generally predictive for small datasets or legacy hardware where the CPU-Memory gap was negligible.

The Hardware Reality: The "Memory Wall"

Modern architectures violate the RAM assumptions due to the **Memory Hierarchy**. There is a significant gap (the "Memory Wall") between the speed of the CPU and the speed of the storage systems.

- **Internal Memory (Cache/RAM):** Fast, small, volatile. Access time in nanoseconds.
- **External Memory (HDD/SSD):** Slow, huge, persistent. Access time in milliseconds.

  The I/O Bottleneck The difference in speed between accessing registers/cache and accessing a disk is a factor of approximately 10^5 **to** 10^6.

  If a CPU cycle is 1 second:

  - Cache access $\approx$ Seconds.
  - RAM access $\approx$ Minutes.
  - Disk access $\approx$ **Months**.

Because of this disparity, the number of CPU instructions executed is often less relevant than the number of times data is moved between disk and memory. We need a model that counts **I/Os** (Input/Output operations).

---

2. The Two-Level Memory Model

To capture the I/O bottleneck, we use the **External Memory Model** (or Disk Access Model / I/O Model). This abstracts the computer into two levels:

1. **Internal Memory:** Limited size M.
2. **External Memory:** Unlimited size.

Key Parameters

We define the system using three fundamental parameters:

- N: The size of the problem instance (number of items).
- M: The size of the **Internal Memory** (number of items that fit in RAM).
- B: The size of a **Disk Block** (number of items transferred in a single I/O).

Rules of the Model

1. Computation can only happen on data present in **Internal Memory**.
2. Data is transferred between Internal and External memory in **blocks** of size B.
3. **Cost Function:** The complexity of an algorithm is the total number of **I/O operations** (or "page faults") performed. We denote this as $\mathcal{C}_{IO}$.
   - CPU time is considered free (or secondary).
   - We assume $1 \ll B \leq M < N$.

---

3. Basic I/O Analysis

3.1 Scanning (Linear Scan)

Consider the problem of summing an array A of N integers stored contiguously on disk.

Algorithm:

1. Read the first block of the array into memory.
2. Process all B items in that block.
3. Discard the block and read the next one.

Analysis:

- **RAM Model Cost:** $O(N)$ (we touch every item).
- **I/O Model Cost:** Since we read data in chunks of B, we perform one I/O for every B items.

$$\text{Cost}_{scan}(N) = \Theta\left(\frac{N}{B}\right) = \Theta\left(\lceil N/B \rceil\right) \text{ I/Os}$$

Note on Spatial Locality: The term N/B highlights the power of **spatial locality**. By grouping data physically close to each other, a single I/O retrieves B useful items, effectively dividing the cost by B. This is the theoretical lower bound for any algorithm that must read the entire input.

Generalized Scanning: The $A(s, b)$ Family We can generalize scanning to account for different access strides.

- Let b be a **logical block size** (smaller than physical block B).
- Let s be a **step size** (jump).
- Algorithm $A(s, b)$: Scan logical blocks A_j jumping by step s.

$$j = (i \cdot s) \mod (N/b)$$

3.2 Searching: Binary Search vs. B-Trees

Consider searching for a specific key k in a sorted array of size N.

Case A: Standard Binary Search In the RAM model, Binary Search is optimal with $O(\log_2 N)$ comparisons. In the I/O model, however, it performs poorly.

1. We look at the middle element. This causes **1 I/O**.
2. We jump to the middle of the left or right subarray. This jump likely lands in a different block, causing **1 I/O**.
3. This continues until the search interval is small enough to fit inside a single block ($\leq B$).

Cost Analysis: The number of levels in the recursion before the interval size drops to B is:

$$\text{Cost}_{binsearch}(N) = O\left(\log_2 \frac{N}{B}\right) \text{ I/Os}$$

Critique: The base of the logarithm is 2. For large N, this is still significant. We are not utilizing the block transfer B effectively; we load B items but only look at 1 (the pivot).

Case B: B-Tree Search (Multi-way Search) To optimize for B, we use a **B-Tree** (or generally, an (a, b)-tree).

- **Node Structure:** Each node is the size of a disk page (B).
- **Fan-out:** A node contains $\Theta(B)$ keys and $\Theta(B)$ pointers to children.

Algorithm:

1. Load the root node (1 I/O).
2. Perform binary search *in memory* over the $\Theta(B)$ keys within that node to find the correct child pointer (0 I/O cost, only CPU time).
3. Load the child node (1 I/O).

Cost Analysis: The height of the tree determines the I/O cost. Since the fan-out is $\Theta(B)$, the height is logarithmic in base B:

$$\text{Cost}_{B-Tree}(N) = O\left(\log_B \frac{N}{B}\right) \approx O\left(\log_B N\right) \text{ I/Os}$$

Comparison:

$$\frac{\text{Binary Search}}{\text{B-Tree}} = \frac{\log_2 N}{\log_B N} = \frac{\ln N / \ln 2}{\ln N / \ln B} = \log_2 B$$

Since B is typically large (e.g., $B = 4096$ bytes ≈ 1000 integers), $\log_2 B \approx 10$. **B-Trees are a factor of $\log_2 B$ faster than Binary Search on disk.**

---

4. Virtual Memory: A Probabilistic Analysis

Why does the OS Virtual Memory system perform poorly when algorithms are not I/O aware? We can model this probabilistically to see the impact of the "penalty" factor.

The Setup

- Let $N = (1 + \epsilon)M$. The dataset is slightly larger than RAM.
- Let c be the **I/O penalty** (the ratio of Disk access time to RAM access time, e.g., $c \approx 10^5$).
- Let a be the probability that an instruction accesses memory (e.g., $a \approx 0.3$).
- Let $p(\epsilon)$ be the probability of a **Page Fault** (data is not in RAM).

Average Step Cost (T_{avg})

The average time to execute one step of the algorithm is the weighted sum of executing it in RAM versus fetching from Disk.

$$T_{avg} = \underbrace{(1-a) \cdot 1}_{\text{CPU ops}} + \underbrace{a \cdot [\underbrace{(1 - p(\epsilon)) \cdot 1}_{\text{Mem Hit}} + \underbrace{p(\epsilon) \cdot c}_{\text{Mem Miss (Fault)}}]}_{\text{Memory Access}}$$

Assuming c is very large, the term $p(\epsilon) \cdot c$ dominates. The formula simplifies to:

$$T_{avg} \approx a \cdot p(\epsilon) \cdot c$$

Interpretation

If we have a standard algorithm (like random access in a Hash Table or Binary Search) where we access memory randomly, $p(\epsilon)$ might be small, but non-negligible.

- **Scenario:** $a = 0.3$, $c = 10^5$.
- **Low Fault Rate:** Even if we only fault once every 1000 accesses ($p(\epsilon) = 10^{-3}$):

  $$T_{avg} \approx 0.3 \cdot 10^{-3} \cdot 10^5 = 30$$

  The algorithm runs **30 times slower** than CPU speed.
- **High Fault Rate:** If $p(\epsilon) \approx 1$ (thrashing), the algorithm runs 10^5 times slower.

This mathematical derivation confirms that minimizing $p(\epsilon)$—by designing algorithms that respect locality (B)—is crucial for performance, far more than optimizing CPU instructions.

External Sorting and Permuting

1. Sorting vs. Permuting in the I/O Model

In the standard RAM model, sorting takes $O(N \log N)$ while permuting (reordering data according to a given permutation π) takes linear time $O(N)$. In the **External Memory Model**, this intuition changes drastically.

1.1 The I/O Complexity

- **Sorting Cost:**
$$\text{Sort}(N) = O\left(\frac{N}{B} \log_{M/B} \frac{N}{M}\right) \text{ I/Os}$$
- **Permuting Cost:**
$$\text{Perm}(N) = O\left(\min\left\{N, \text{Sort}(N)\right\}\right) \text{ I/Os}$$

1.2 The Permuting Bottleneck

Permuting is considered an **I/O bottleneck**.

- If we naively follow the permutation π to move elements one by one:
  1. Read index i.
  2. Fetch element from position $\pi[i]$.
  3. If $\pi[i]$ is in a different block than the previous element, we pay **1 I/O**.
  4. **Worst Case:** Every access is a cache miss. Total Cost $= O(N)$ I/Os.
- **Lack of Spatial Locality:** Unlike scanning (N/B), we lose the block advantage because the target locations are scattered randomly across the disk.

Permuting via Sorting: The Algorithm We can reduce the permuting problem to sorting to achieve the lower I/O cost. Given vector S (data) and P (target positions), where $S[i]$ should move to position $P[i]$:

1. **Scan** S and write pairs $\langle \text{item}, \text{old_pos} \rangle$.
2. **Scan** P and write pairs $\langle \text{new_pos}, \text{old_pos} \rangle$.
3. **Sort** the P pairs by their second component (old_pos).
4. **Scan** the sorted P and S together (merge). Since they are both ordered by old_pos, we can link $\langle \text{item} \rangle$ with $\langle \text{new_pos} \rangle$. Create pairs $\langle \text{item}, \text{new_pos} \rangle$.
5. **Sort** these new pairs by their second component (new_pos).
6. **Scan** and extract just the items.

Total Cost: 4 Scans + 2 Sorts $\approx O(\text{Sort}(N))$.

---

2. MergeSort Analysis

MergeSort is the standard approach for external sorting. We divide the file into chunks that fit in memory, sort them (creating "runs"), and then merge them.

2.1 Binary MergeSort

This is the standard 2-way merge adapted for disk.

1. **Run Generation:** Create N/M sorted runs.
2. **Merging:** Merge 2 runs at a time.

Analysis:

- Number of Passes (Recursion Depth): $\log_2(N/M)$.
- Cost per Pass: We read and write the entire dataset once per level $\rightarrow 2 \cdot (N/B)$.

- **Total Cost:**

$$\text{Cost}_{Binary} = O\left(\frac{N}{B}\log_2\frac{N}{M}\right) \text{ I/Os}$$

- *Critique:* The base 2 is inefficient. We are not using the available memory M to merge more than 2 streams at once.

2.2 Multi-way MergeSort (k-way Merge)

To optimize, we maximize the number of streams we merge simultaneously.

The Algorithm:

1. **Setup:** We have memory M and block size B.
2. **Fan-in:** We can hold 1 block from k different input streams and 1 block for the output stream in memory.

$$k \cdot B + 1 \cdot B \leq M \implies k \approx \frac{M}{B} - 1$$

3. **Process:** Use a **Min-Heap** of size k in memory to efficiently select the smallest item among the k streams.
   - CPU Time per item: $O(\log k)$.
   - I/O behavior: When an input buffer empties, load the next block (1 I/O). When the output buffer fills, write it to disk (1 I/O).

Rigorous Complexity Derivation:

- **Initial Runs:** N/M runs (or $N/2M$ with Snow Plow).
- **Reduction Factor:** In each pass, we reduce the number of runs by a factor of $k \approx M/B$.
- **Number of Passes:**

$$\text{Passes} = \log_{M/B}\left(\frac{N}{M}\right)$$

- **Total Cost:**

$$\text{Cost}_{Multi-way} = O\left(\frac{N}{B}\log_{M/B}\frac{N}{M}\right) \text{ I/Os}$$

  Why Base M/B? The base M/B represents the **fan-in** of the merge process. We leverage the entire memory M to buffer blocks B. This is significantly faster than binary merge ($\log_2$ vs $\log_{M/B}$).

---

3. Run Generation: The "Snow Plow"

Instead of just reading M items, sorting them, and writing them out (producing runs of length M), we can use the "Snow Plow" technique to generate runs of average length $2M$.

3.1 The Analogy

Imagine a snow plow moving around a circular track (memory). Snow (data) falls continuously. The plow pushes out snow that fits the current order, but snow that falls "behind" the plow (smaller than current max) stays for the next round.

3.2 The Algorithm

We partition memory M into a **Min-Heap (H)** and an **Unsorted Buffer (U)**. Initially, H is full (size M), U is empty.

1. **Extract:** Remove min element m from H and write to Output.
2. **Read:** Read next element x from Input.

3. **Compare & Insert:**
   - **Case A ($x \geq m$):** x can be part of the *current* run. Insert x into H.
   - **Case B ($x < m$):** x is smaller than what we just wrote; it cannot be in the current sorted run. Place x into U (it is "dead" for now).
4. **Heap Shrinks:** If we put x in U, the effective size of H decreases.
5. **Restart:** When H is empty (memory is full of "dead" elements in U), the current run ends. Move all U to H, rebuild Heap, start new run.

3.3 Average Run Length Proof

- **Intuition:** Assume input is random.
- When we read an item x, $P(x < m) \approx 1/2$.
- So, for every 2 items we read, 1 goes to H (extending the run) and 1 goes to U (stored for next run).
- We start with M items. To fill U with M items (ending the phase), we must have processed roughly $2M$ items total.
- **Result:** Average run length $= 2M$.
- *Benefit:* Halves the number of initial runs, saving 1 merge pass in some cases.

---

4. Disk Striping (D Disks)

To increase bandwidth, we use D independent disks.

4.1 Striping Technique

- **Concept:** Distribute data cyclically across disks (Block 0 on Disk 0, Block 1 on Disk 1, etc.).
- **Logical Block:** We treat the D disks as a single large disk with a **Logical Block Size** of:

$$B' = D \times B$$

- **Memory Constraint:** We need enough memory to buffer one block from every disk:

$$M \geq D \times B$$

4.2 Complexity with Striping

We substitute B with $D \cdot B$ in the Multi-way MergeSort formula:

$$\text{Cost}_{Striped} = O\left(\frac{N}{D \cdot B} \log_{M/(D \cdot B)} \frac{N}{M}\right) \text{ I/Os}$$

4.3 Comparison vs Theoretical Lower Bound

The theoretical lower bound for sorting with D disks is slightly better than what Disk Striping achieves.

$$\text{Ratio} = \frac{\text{Disk Striping}}{\text{Lower Bound}} \approx \frac{\log(M/B)}{\log(M/B) - \log D}$$

- **Interpretation:** Disk Striping is asymptotically slower, but as $M \to \infty$, the ratio approaches 1. It is efficient enough for practical purposes.

---

5. Lower Bounds

Is Multi-way MergeSort optimal?

5.1 The Lower Bound Formula

The theoretical lower bound for sorting N items on D disks is:

$$\Omega\left(\frac{N}{D \cdot B} \log_{M/B} \frac{N}{D \cdot B}\right) \text{ I/Os}$$

5.2 Optimality Check

1. **Single Disk ($D = 1$):**
   - Algorithm: $\frac{N}{B} \log_{M/B} \frac{N}{M}$
   - Lower Bound: $\frac{N}{B} \log_{M/B} \frac{N}{B}$
   - *Verdict:* **Optimal.** (Since $\frac{N}{M}$ and $\frac{N}{B}$ are close enough inside the log).
2. **Multiple Disks ($D > 1$):**
   - Algorithm (Striping): Base is $M/(D \cdot B)$.
   - Lower Bound: Base is M/B.
   - *Verdict:* **NOT Optimal.** Disk Striping reduces the fan-out of the merge because the logical block B' consumes more memory slots.
   - *Note:* To achieve optimality for $D > 1$, complex algorithms like **GreedSort** are required (not covered in standard implementation).

Quicksort and Selection

1. In-Memory Quicksort Variants

While standard Quicksort is $O(N \log N)$ on average, optimizations are crucial for handling pathological cases (duplicates) and modern hardware (cache efficiency).

1.1 3-Way Partitioning (Bentley-McIlroy)

Standard Quicksort suffers when the input has many **duplicate keys**, potentially degrading to $O(N^2)$. The 3-way partition divides the array $S[i, j]$ into three segments relative to the pivot P:

1. **Less than P:** $S[i, l-1]$
2. **Equal to P:** $S[l, r-1]$
3. **Greater than P:** $S[r, c-1]$

Algorithm Logic:

- Use pointers to swap elements into their regions.
- **Crucial Optimization:** The central segment (elements equal to P) is **excluded** from recursive calls.
- If the array consists of all equal keys, this step runs in $O(N)$ and terminates immediately, whereas standard Quicksort would recurse indefinitely.

1.2 Dual-Pivot Quicksort (Yaroslavskiy)

The default algorithm in Java 7+ and Python. It uses **two pivots** (p, q with $p < q$) to partition the array into three regions:

1. $x < p$
2. $p \leq x \leq q$
3. $x > q$

Performance Paradox:

- Theoretically, Dual-Pivot requires **more comparisons** ($1.9N \ln N$) than single-pivot ($1N \ln N$).

- However, it is practically **faster** (often $> 10\%$).
- **Reason:** It minimizes **Branch Mispredictions**. Modern CPUs pipeline instructions; a random conditional jump (like in Quicksort partition) clears the pipeline. Dual-Pivot reduces the number of memory accesses and behaves better with CPU caches, making the "cost per comparison" lower.

1.3 Bounded Quicksort

Standard Quicksort is recursive. In the worst case (unbalanced partitions), the recursion depth can reach $O(N)$, causing a **Stack Overflow**.

The Solution: To guarantee $O(\log N)$ stack space, we rely on **Tail Recursion Elimination** on the larger sub-problem.

Pseudocode:

```
BoundedQS(S, i, j) {
    while (j - i > n0) { // Small subarrays use Insertion Sort
        // 1. Pivot Selection & Partitioning
        p = partition(S, i, j);

        // 2. Identify smaller half
        if (p <= (i + j) / 2) {
            // Left side is smaller: Recurse Left
            BoundedQS(S, i, p - 1);
            i = p + 1; // Iterate on Right
        } else {
            // Right side is smaller: Recurse Right
            BoundedQS(S, p + 1, j);
            j = p - 1; // Iterate on Left
        }
    }
    InsertionSort(S, i, j);
}
```

Since the smaller part is at most half the size, the recursion depth cannot exceed $\log_2 N$.

---

2. The Selection Problem

Problem: Find the k-th smallest element in an unsorted sequence S.

2.1 Complexity Landscape

- **Sorting:** Sort S and pick index k. Time: $O(N \log N)$.
- **QuickSelect:** Randomized partitioning. Time: $O(N)$ expected.

Heap-based Selection ($O(N \log k)$) 1

Useful when k is small relative to N.

1. Maintain a **Max-Heap** of size k.
2. Fill it with the first k elements.
3. Scan the rest of the array ($N - k$ items).
4. If a new item $x <$ Heap.Max(), remove Max and insert x.
5. **Result:** The Max of the heap is the k-th smallest item.

2.2 Randomized Selection (QuickSelect)

Similar to Quicksort, but we only recurse on **one** side.

Algorithm:

1. Pick a random pivot.
2. Partition S into $S_<, S_=, S_>$.
3. If $k \leq |S_<|$, recurse on $S_<$.
4. Else if $k \leq |S_<| + |S_=|$, return Pivot.
5. Else, recurse on $S_>$ seeking rank $k - (|S_<| + |S_=|)$.

Proof of Linear Expected Time:

- A "Good Selection" occurs if the pivot lands in the middle third of the sorted sequence (ranks $[N/3, 2N/3]$).
- This guarantees neither $|S_<|$ nor $|S_>|$ exceeds $2N/3$.
- Probability of Good Selection $= 1/3$.
- Recurrence for expected time $\hat{T}(N)$:

  $$\hat{T}(N) \leq O(N) + \hat{T}\left(\frac{2N}{3}\right)$$

  $$$$By the Master Theorem, this sums to a geometric series dominated by the first term:

  $$\hat{T}(N) = O(N)$$

  $$$$
- **I/O Complexity:** Since partitioning is a scan, Cost $= O(N/B)$ I/Os.

---

3. Multi-way Quicksort (External Memory)

While Multi-way MergeSort is a **bottom-up** (merge) approach, Multi-way Quicksort is a **top-down** (distribution) approach.

3.1 Algorithm Design

1. **Distribution:** Select $k - 1$ pivots to divide the input range into k buckets (partitions).
2. **Scanning:** Read the input sequence. For every element, determine which bucket it belongs to and write it to the corresponding buffer.
   - **Constraints:** We need k output buffers (size B) in memory. Thus, $k \approx M/B$.
3. **Recursion:** Recursively sort each bucket. If a bucket fits in memory ($< M$), load it and sort it internally.

3.2 Pivot Selection via Oversampling

The critical flaw of Quicksort is unbalanced partitions. In external memory, a bad partition wastes entire I/O passes. We need "perfect" pivots that split N items into k buckets of size $\approx N/k$.

The Oversampling Technique:

1. Draw a random sample of size s from the dataset.
2. Sort the sample.
3. Pick elements at regular intervals (s/k) to be the $k-1$ pivots.

Theorem (Sample Size):

To ensure that no bucket exceeds size $4N/k$ with probability $\geq 1/2$, we need an oversampling factor a such that:

$$a + 1 = \frac{1}{2} \ln k$$

Total sample size:

$$s = (a+1)k - 1 \approx \frac{k}{2} \ln k$$

Note: The sample size depends on k (number of buckets) and $\ln k$.

3.3 I/O Complexity Analysis

- **Fan-out:** $k \approx M/B$.
- ***Cost per Level:** We read and write the whole dataset once: $2(N/B)$.
- **Number of Levels:** The recursion depth is $\log_k(N/M)$.

$$\text{Cost}Multi - Quick = O\left(\frac{N}{B} \log M/B \frac{N}{M}\right) \text{ I/Os}$$

This matches the sorting lower bound.

3.4 Comparison: MergeSort vs Quicksort

Feature	Multi-way MergeSort	Multi-way Quicksort
Paradigm	Merging (Bottom-Up)	Distribution (Top-Down)
I/O Access	Sequential Read / Sequential Write	Sequential Read / **Random** Write (to k buffers)
Space	Easy to manage	Harder (need robust pivots)
Preference	Generally preferred for stability and guaranteed balance.	Preferred if random writes are fast or for parallel systems.

Random Sampling and Streams

Sampling is fundamental when datasets are too massive to process entirely. We analyze two scenarios: one where the dataset size N is known and stored on disk, and one where data arrives as a **Stream** of unknown length.

---

1. Disk Sampling (Known Length N)

1.1 Algorithm A: Iterative Scanning

We want to select exactly s items from a file of size N.

The Algorithm: Iterate through the file item by item (from $j = 0$ to $N - 1$). Let:

- t: number of items scanned so far.
- m: number of items *already selected* so far.
- s: total items we want to select.

For the current item x, we select it with probability: $$P(\text{select } x) = \frac{s - m}{N - t}

$$

Logic:

- The numerator $(s - m)$ is the number of items we *still need* to pick.
- The denominator $(N - t)$ is the number of items *remaining* in the file.
- This probability dynamically adjusts. If we haven't picked enough items recently, the probability increases. If we have picked many, it decreases.

Correctness: This guarantees that every item has an equal probability s/N of being selected.

I/O Efficiency:

- **Pros:** It is a single sequential scan.
- **Cons:** We must read the entire file (Cost $= N/B$). If $s \ll N$, this is inefficient. We would prefer an algorithm that skips blocks, but skipping requires random access, which is expensive on disk unless s is very small.

1.2 Algorithm B: Dictionary-based Sampling

If we need to sample m items where $m \ll N$, and we have random access:

1. Generate a random index $p \in [1, N]$.
2. Check if p is in a dictionary D (BST or Hash Table).
3. If not, insert p into D and pick item $S[p]$.
4. Repeat until $|D| = m$.

Complexity:

- Expect to try roughly m times (collisions are rare if $m \ll N$).
- Cost: $O(m \log m)$ insertions into Dictionary (BST).
- I/O: $O(m)$ random accesses.

---

2. Streaming Model & Reservoir Sampling (Unknown Length)

2.1 The Streaming Model

- **Constraint:** Data arrives as a sequence $x_1, x_2, \ldots$ and we see each item only once.
- **Unknown N:** We do not know when the stream ends.
- **Goal:** At any point in time t, we want to maintain a representative random sample of size s of all t items seen so far.

2.2 Reservoir Sampling Algorithm (Waterman's Algorithm)

We maintain a "Reservoir" (buffer) R of size s.

Algorithm:

1. **Initialization:** Put the first s items of the stream directly into R.
2. **Processing item t (where $t > s$):**

- Generate a random number $h \in [1, t]$.
- If $h \leq s$:
- **Swap:** Replace the item at index $R[h]$ with the new item x_t.
- Else ($h > s$):
- **Discard:** Ignore x_t.

Probability Logic: The probability of accepting the t-th item into the reservoir is: $$P(\text{keep } x_t) = \frac{s}{t}

$$

2.3 Proof of Uniformity

We must prove that after seeing t items, **every** item seen so far ($x_1 \dots x_t$) has an equal probability $\frac{s}{t}$ of being in R.

Proof by Induction:

Base Case ($t = s$): Every item is in the reservoir with probability $1 = s/s$. Correct.

Inductive Step: Assume that after $t - 1$ items, every item x_i (where $i < t$) is in R with probability $\frac{s}{t-1}$. Now consider the arrival of item x_t.

1. **For the new item x_t:** By definition, it is selected with probability $\frac{s}{t}$. Correct.
2. **For an old item x_i ($i < t$) already in R:** x_i remains in R if it is **not** replaced by x_t.

x_i is removed only if:

- x_t is selected (Prob $= \frac{s}{t}$).
- **AND** the random index h chosen for replacement is exactly the index of x_i (Prob $= \frac{1}{s}$).

So, Prob(x_i removed) $= \frac{s}{t} \times \frac{1}{s} = \frac{1}{t}$.

Therefore, Prob(x_i survives) $= 1 - \frac{1}{t} = \frac{t-1}{t}$.

Total Probability:

$$P(x_i \in R \text{ at } t) = P(x_i \in R \text{ at } t - 1) \times P(x_i \text{ survives})$$

Substitute the inductive hypothesis:

$$P(x_i \in R \text{ at } t) = \frac{s}{t-1} \times \frac{t-1}{t} = \frac{s}{t}$$

Conclusion: After step t, every item has probability s/t of being in the reservoir.

Intersection Algorithms

The "Intersection Problem" is a canonical operation in Search Engines (handling "AND" queries between inverted lists). **Problem:** Given two sorted lists L_1 (length n) and L_2 (length m), with $n \leq m$, return $L_1 \cap L_2$.

---

1. Basic Approaches

1.1 Merge-based Intersection

The standard approach is to place a pointer at the start of each list and advance the pointer pointing to the smaller value.

- **Algorithm:**
  - If $L_1[i] < L_2[j]$, advance i.
  - If $L_1[i] > L_2[j]$, advance j.
  - If $L_1[i] == L_2[j]$, add to result, advance both.
- **Complexity:** $O(n + m)$.
- **Critique:** This is optimal when $n \approx m$. However, if $n \ll m$ (e.g., searching for "Quicksort" [rare] AND "The" [common]), we waste time scanning the entirety of the long list L_2 ($O(m)$) just to verify n items.

1.2 Binary Search-based

We iterate through the short list L_1 and, for every element $x \in L_1$, perform a binary search in L_2.

- **Complexity:** $O(n \log m)$.
- **Comparison:** Better than Merge if $n \log m < m$.

---

2. Advanced Intersections

2.1 Doubling Search (Galloping)

This algorithm adapts to the distribution of matches. It avoids searching the entire remaining part of L_2 by estimating where the next match *could* be.

The Algorithm: Let i be the current position in L_2 (after finding the previous match). To find the next element $x \in L_1$ in L_2:

1. **Gallop:** Check indices $i + 1, i + 2, i + 4, \ldots, i + 2^k$ in L_2.
2. **Stop:** Stop when we find a window where $L_2[i + 2^{k-1}] < x \leq L_2[i + 2^k]$.
3. **Binary Search:** Perform standard binary search only within this specific window (size 2^{k-1}).

Complexity Analysis: Let d_j be the distance in L_2 between the $(j - 1)$-th match and the j-th match. The cost to find the j-th element is $O(\log d_j)$. Total cost:

$$\text{Cost} = \sum_{j=1}^{n} O(\log d_j)$$

Since $\sum d_j \leq m$ (the total length of L_2), by Jensen's inequality (concavity of log), this sum is maximized when all d_j are equal ($d_j \approx m/n$).

$$\text{Total Cost} = O\left(n\left(1 + \log \frac{m}{n}\right)\right)$$

This is asymptotically optimal for comparison-based intersection.

2.2 Mutual Partitioning

A recursive, divide-and-conquer approach.

1. Pick the **median** of the shorter list L_1. Let this be p.
2. Binary search for p in L_2. This splits both lists into (Left, Right) sets.
3. **Recurse:** Solve Intersect($L_{1,left}, L_{2,left}$) and Intersect($L_{1,right}, L_{2,right}$).
4. *Swap roles:* In the recursive calls, if the "Left" part of L_2 becomes shorter than L_1's part, swap them so we always iterate/pivot on the shorter list.

- **Complexity:** $O(n(1 + \log(m/n)))$. Matches the Galloping bound.

2.3 Two-Level Memory Approach (Cache Blocking)

Standard algorithms ignore the cache. A simple scan might fetch a cache line (block L) but only use 1 item if the intersection is sparse. **Goal:** Perform intersection at the granularity of cache blocks.

Setup:

1. Partition arrays into blocks of size L (cache line size).
2. Create a meta-array A' containing the *first key* of every block in A. $|A'| = n/L$.

Algorithm:

1. **Filter:** Intersect B with the small meta-array A'. This identifies which blocks in A *might* contain elements from B.
2. **Refine:** Only load and intersect the specific blocks A_j that passed the filter.

Complexity:

$$\text{Cost} \approx O\left(\frac{n}{L} + m + m \cdot L\right)$$

- n/L: Scanning the meta-array.
- $m \cdot L$: Worst case, every item in B falls into a different block of A, forcing us to scan a full block of size L for each item.

---

3. Interpolation Search

Standard Binary Search blindly cuts the array in half ($mid = (low + high)/2$). **Interpolation Search** acts like humans searching a phonebook: if looking for "Zuck", we open near the end; if "Ada", near the beginning.

3.1 The Algorithm

We estimate the position of target x using the formula:

$$next = low + \lfloor \frac{x - A[low]}{A[high] - A[low]} \times (high - low) \rfloor$$

- **Logic:** It assumes a linear relation between the value of keys and their array indices.

3.2 Assumptions & Complexity

- **Requirement:** Keys must be drawn from a **Uniform Distribution**.
- **Performance:**
  - **Average Case:** $O(\log \log N)$.
  - **Worst Case:** $O(N)$ (if distribution is highly skewed, e.g., exponential).
- **Derivation:** The recurrence relation roughly follows $T(N) \approx T(\sqrt{N}) + O(1)$, which resolves to $\log \log N$.

3.3 Why it fails on Disk

Interpolation Search is terrible for external memory.

1. **Calculation:** The formula involves division/multiplication (CPU heavy, though negligible compared to I/O).
2. **Random Access:** The `next` position jumps unpredictably. It does not narrow down to a specific block B quickly enough to benefit from caching.
3. **Data Requirements:** Real-world data is rarely perfectly uniform.

Conclusion: For in-memory uniform data, Interpolation Search is fast. For disk-based or skewed data, B-Trees or Galloping Search are superior.

Randomized Dictionaries

In this chapter, we explore dictionary data structures that rely on **randomization** to achieve balance, rather than complex deterministic rebalancing rules (like AVL or Red-Black trees).

---

1. Skip Lists

A **Skip List** is a probabilistic alternative to balanced trees. It consists of sorted linked lists arranged in levels.

1.1 Structure and Levels

- **Level L_0:** Contains all elements in sorted order.
- **Level L_i:** Contains a subset of elements from L_{i-1}.
- **Sentinels:** All levels start with $-\infty$ and end with $+\infty$.

Promotion Logic (The Coin Flip): When inserting an element x:

1. Insert x into L_0.
2. Flip a fair coin ($p = 0.5$).
   - **Heads:** Promote x to level L_1. Flip again.
   - **Tails:** Stop promoting.
3. Repeat until Tails occurs.

Biased Skip Lists Standard Skip Lists assume uniform access. If some keys are accessed more frequently, we want them higher up (shorter search path).

- **Idea:** Unlike the random coin flip, we use a **deterministic stack** based on access probability $p(x)$.
- **Height:** We force the height of key x to be proportional to $\log(1/p(x))$.
- **Result:** Frequent items stay near the top, behaving like a static optimal search tree.

1.2 Complexity Analysis

Space Analysis Let N be the number of elements.

- Prob(x exists at level i) $= 1/2^i$.
- Expected size of level i: $N/2^i$.
- Total Expected Space:

$$\sum_{i=0}^{\infty} \frac{N}{2^i} = N \sum_{i=0}^{\infty} \left(\frac{1}{2}\right)^i = N \cdot 2 = O(N)$$

Search Time Analysis

- **Height (H):** The probability that a Skip List has height greater than $c \log N$ is extremely low ($1/N^{c-1}$). Thus, $H = O(\log N)$ w.h.p.
- **Search Path:** We start at the top-left. At any node:
  - If `next.key` $\leq$ target: Move **Right**.
  - Else: Move **Down**.
- **Expected Steps:** The number of steps is proportional to the height.

$$E[\text{Search Time}] = O(\log N)$$

1.3 I/O Analysis (The Weakness)

Skip Lists are **not** I/O efficient compared to B-Trees.

- **Pointer Chasing:** Moving "Right" or "Down" involves following a pointer to a potentially random memory location (or disk block).
- **No Blocking:** Unlike a B-Tree node, which aggregates B keys into one block, a Skip List node contains only one key and pointers.
- **I/O Cost:** In the worst case (cold cache), a search involves $O(\log N)$ I/Os.
  - Compare to B-Trees: $O(\log_B N)$.
  - Since $B \approx 1000$, $\log_B N$ is vastly smaller than $\log_2 N$.

---

2. Treaps (Tree + Heap)

A **Treap** is a binary tree that satisfies two properties simultaneously:

1. **BST Property (on Keys):** For any node u, keys in the left subtree are $< key(u)$, and keys in the right subtree are $> key(u)$.
2. **Heap Property (on Priorities):** For any node u, $priority(parent(u)) < priority(u)$ (Min-Heap variant).

2.1 Randomization & Balance

- **Priorities:** When a node is inserted, it is assigned a unique priority chosen uniformly at random.
- **Theorem:** A Treap with random priorities is isomorphic to a **Random Binary Search Tree (RBST)** (a BST constructed by inserting keys in a random permutation).
  - *implication:* The expected height is $O(\log N)$.

2.2 Rotations

Insertions are performed as standard BST insertions (placing the node as a leaf). This may violate the Heap property. **Fix:** Perform **Rotations** to bubble the node up until the Heap property is restored. Rotations preserve the BST ordering of keys.

2.3 Operations

Split(T, k) Divides Treap T into two Treaps: $T_{\leq}$ (keys $\leq k$) and $T_{>}$ (keys $> k$).

- **Algorithm:**
  1. Insert a "dummy" node with key k and priority $-\infty$.
  2. Because priority is minimal, it bubbles up to the **root** via rotations.
  3. The left child of the root is $T_{\leq}$, the right child is $T_{>}$.
  4. Remove the dummy node.
- **Complexity:** $O(\log N)$.

Merge(T_1, T_2) Joins two Treaps (assuming all keys in $T_1 <$ all keys in T_2).

- **Algorithm:**
  1. Create a dummy root with priority $-\infty$.
  2. Attach T_1 as left child, T_2 as right child.
  3. "Sink" the dummy root down (rotating with the child having higher priority) until it becomes a leaf.
  4. Delete the dummy leaf.
- **Complexity:** $O(\log N)$.

Delete(k) Deletion is the inverse of insertion.

1. Find node u with key k.
2. Set priority of u to $+\infty$ (for Min-Heap) or $-\infty$ (for Max-Heap).
3. **Rotate Down:** Swap u with its child having the higher/lower priority (depending on Heap type) to maintain Heap property locally.
4. Repeat until u is a leaf.
5. Cut u.

2.4 3-Sided Range Query

Query: Find all nodes (x, y) such that $q_1 \leq x \leq q_2$ and $y \leq q_3$ (Range on Key, Threshold on Priority).

Algorithm:

1. **Search Spines:** Find the paths to q_1 and q_2 in the tree.
2. **Identify Subtrees:** Identify the subtrees hanging "between" these two paths.
3. **Pruning:** For each candidate subtree:
   - Check the root's priority.
   - If $root.prio > q_3$: **Stop**. By the Heap property, no node in this subtree can satisfy the condition.
   - If $root.prio \leq q_3$: Report root and recurse.

- **Complexity:** $O(\log N + K)$ where K is the number of reported items.

---

3. Proof of Average Depth

We prove that the expected depth of a node in a Random BST (Treap) is $O(\log N)$.

Setup: Let keys be $x_1 < x_2 < \cdots < x_N$. Define indicator variable $A_{i,k}$:

$$A_{i,k} = \begin{cases} 1 & \text{if } x_i \text{ is an ancestor of } x_k \\ 0 & \text{otherwise} \end{cases}$$

Depth Formulation: The depth of node x_k is the number of its ancestors:

$$D_k = \sum_{i=1}^{N} A_{i,k}$$

By Linearity of Expectation:

$$E[D_k] = \sum_{i=1}^{N} P(x_i \text{ is ancestor of } x_k)$$

Ancestor Condition: In a Treap, x_i is an ancestor of x_k **if and only if** x_i has the **lowest priority** among all nodes in the range between them (inclusive). Let this range be $R_{ik} = \{\min(i,k), \ldots, \max(i,k)\}$.

- Size of range: $|k - i| + 1$.
- Since priorities are random, every node in R_{ik} is equally likely to be the minimum.

$$P(x_i \text{ is ancestor of } x_k) = \frac{1}{|k-i|+1}$$

Summation:

$$E[D_k] = \sum_{i=1}^{N} \frac{1}{|k-i|+1} = \underbrace{\sum_{j=1}^{k} \frac{1}{j}}_{\text{Left side}} + \underbrace{\sum_{j=1}^{N-k+1} \frac{1}{j}}_{\text{Right side}} - 1$$

These are **Harmonic Numbers** ($H_n \approx \ln n$).

$$E[D_k] \approx \ln k + \ln(N-k) \leq 2 \ln N \approx 1.39 \log_2 N$$

Conclusion: Expected depth is $O(\log N)$.

String Sorting and Tries

Sorting strings differs fundamentally from sorting atomic keys (like integers) because strings have variable lengths, and comparisons depend on prefixes.

---

1. String Sorting

1.1 The Challenge

Standard comparison-based sorting (MergeSort, HeapSort) has a lower bound of $\Omega(n \log n)$ **comparisons**. However, for strings, a single comparison is not $O(1)$. Comparing two strings s_1, s_2 takes $O(|LCP(s_1, s_2)|)$ time.

- **Total naive cost:** $O(L_{avg} \cdot n \log n) = O(N \log n)$, where N is the total number of characters.
- **Optimality:** This is suboptimal. We end up re-scanning long common prefixes repeatedly.
- **True Lower Bound:** $\Omega(d + n \log n)$, where d is the sum of lengths of **distinguishing prefixes** (the shortest prefix needed to distinguish each string from the others).

1.2 Multi-key Quicksort (3-way String Quicksort)

An elegant adaptation of Quicksort for variable-length strings.

Algorithm: Given a set R and current character index i (initially 0):

1. Pick a pivot string p. Let $c = p[i]$ be the pivot character.
2. **3-way Partition** R into:
   - $R_<$: Strings where $s[i] < c$.
   - $R_=$: Strings where $s[i] = c$.
   - $R_>$: Strings where $s[i] > c$.
3. **Recurse:**
   - `MKQS`$(R_<, i)$ (Pivot character was not matched, stay at index i).
   - `MKQS`$(R_>, i)$ (Pivot character was not matched, stay at index i).
   - `MKQS`$(R_=, i+1)$ (**Crucial:** We matched character i, so advance to $i+1$).

Complexity:

- **Time:** $O(d + n \log n)$. Matches the lower bound.
- **Why?** Characters in identifying prefixes (d) are participating in the "=" partition. Other comparisons contribute to the sorting cost ($n \log n$).

1.3 Radix Sorts

LSD (Least Significant Digit) Sorts from the last character to the first.

- **Requirement:** The inner sorting algorithm must be **Stable** (e.g., Counting Sort). If it weren't stable, sorting by character i would scramble the order established by character $i+1$.
- **Complexity:** $O(L_{max} \cdot n)$.
- **Drawback:** Must touch every character, even if strings are distinct by the first letter.

MSD (Most Significant Digit) Sorts from the first character to the last (recursive buckets).

- **Variable Lengths:** Shorter strings are conceptually padded with a sentinel character (smaller than any alphabet char) to ensure they end up in the correct bucket.
- **Complexity:** $O(N)$ in practice, but overhead of recursion is high for small buckets.

---

2. Trie-based Structures

2.1 Standard Tries (Uncompacted)

A tree where edges are labeled with characters.

- **Node Structure:** An array of size σ (alphabet size).
- **Problem:** Space complexity is $O(N \cdot \sigma)$. Most nodes in a Trie have only 1 child (sparse), wasting huge amounts of memory for large alphabets (like Unicode).

2.2 Ternary Search Trees (TST)

Combines the time efficiency of Tries with the space efficiency of BSTs.

- **Node Structure:**
  - `char c` (Split Character)
  - `left` pointer ($< c$)
  - `eq` pointer ($= c$)
  - `right` pointer ($> c$)
- **Traversal:** Similar to Multi-key Quicksort. If matches `c`, follow `eq`. Else, follow `left/right`.
- **Space:** $O(N)$ nodes. No dependency on σ.

2.3 Compacted Tries (Patricia Tries)

Optimizes standard Tries by compressing paths of single-child nodes.

- **Compression:** A chain of nodes `a -> p -> p -> l -> e` is collapsed into a single edge labeled "apple".
- **Edge Label:** Stored as a triple $\langle \text{string_id}, \text{start}, \text{end} \rangle$.
- **Node Count:** Guaranteed $O(n)$ nodes (where n is number of strings), regardless of total length N. Every internal node has branching factor ≥ 2.

Patricia Trie Search: The 3 Phases Patricia Tries store only the first char of the edge label + the label length (to skip). This creates ambiguity. **Algorithm:**

1. **Blind Downward Traversal:** Follow edges matching the single character. Skip the number of characters indicated by the edge length. Do *not* check the skipped characters (blind).
2. **Leaf Selection:** We eventually hit a leaf (or fail). Let the leaf be string S.
3. **Upward Verification:** Calculate $LCP(P, S)$. If $LCP == |P|$, we found it. If not, the mismatch character determines the lexicographic relationship.

- *Why?* This avoids decoding the edge labels (pointers) during the traversal, reducing cache misses.

2.4 Array-based Solutions

- **Naive Array of Pointers:** `malloc` for every string. Causes massive cache misses (pointer chasing).
- **Giant Block:** Concatenate all strings into one large contiguous memory block (separated by `\0`). The "pointer" array becomes an array of integer offsets.
  - Improves locality significantly.
  - Allows pointer compression (offsets are smaller than 64-bit pointers).

2.5 Front Coding

Used for storing sorted dictionaries on disk (e.g., inside B-Tree leaves).

- **Idea:** Sorted strings share long prefixes.
- **Format:** Store shared prefix length ℓ + remaining suffix.
  - `alcatraz`
  - `alcohol` → `(2, cohol)` (matches 'al')
  - `alcoholic` → `(5, lic)` (matches 'alcoh')

Full Text Indexing

Full Text Indexing solves the problem of finding a pattern P in a text T efficiently. While Suffix Trees are powerful, they are space-heavy. **Suffix Arrays (SA)** combined with the **LCP Array** offer a space-efficient alternative that supports similar operations.

---

1. Suffix Array (SA)

1.1 Definition

Given a text T of length N, the Suffix Array `SA` is an array of integers from 0 to $N-1$.

- **Content:** `SA[i]` is the starting position of the i-th lexicographically smallest suffix of T.
- **Space:** N integers ($4N$ bytes), which is much smaller than a Suffix Tree pointer structure.

1.2 Binary Search on SA ($O(|P| \log N)$)

Since `SA` stores suffixes in sorted order, all suffixes starting with pattern P appear contiguously in `SA`. We can find the range $[L, R]$ of these suffixes using Binary Search.

Algorithm:

1. **Compare:** Compare P with the suffix starting at `SA[mid]`.
2. **Branch:**
   - If $P < T[\text{SA}[\text{mid}]]$, go Left.
   - If $P > T[\text{SA}[\text{mid}]]$, go Right.
3. **Repeat:** Standard binary search steps.

Complexity:

- Number of steps: $O(\log N)$.
- Cost per step: Comparison takes $O(|P|)$ in worst case.
- **Total:** $O(|P| \log N)$.

1.3 LCP-LR Optimization ($O(|P| + \log N)$)

We can speed this up by avoiding redundant comparisons. We precompute two auxiliary arrays L_{lcp} and R_{lcp} for the binary search ranges.

- Let $l = LCP(P, \text{suffix}_{Low})$.
- Let $r = LCP(P, \text{suffix}_{High})$.
- When checking `mid`, we know the shared prefix between `Low` and `mid` (stored in L_{lcp}).
  - If $L_{lcp}[mid] > l$, then P matches `mid` up to l automatically, and the mismatch is further down.
  - If $L_{lcp}[mid] < l$, then P is determined by the mismatch character known from `Low`.
- **Result:** We only advance the character pointer on P (never backtrack). Total comparisons limited to $O(|P|)$.

- **Total:** $O(|P| + \log N)$.

---

2. LCP Array Construction (Kasai's Algorithm)

The **Longest Common Prefix (LCP)** array stores the length of the shared prefix between consecutive suffixes in the sorted SA.

$$LCP[i] = \text{length}(LCP(\text{suffix}_{SA[i]}, \text{suffix}_{SA[i-1]}))$$

2.1 The Algorithm (Linear Time)

Naive construction takes $O(N^2)$. Kasai's algorithm achieves $O(N)$ by iterating through suffixes in **text order** ($i = 0 \dots N$), not SA order.

Comparison: Brute Force vs Kasai Example $T =$ "banana$".

- **Brute Force:** Compares every pair from scratch. Total comparisons can be quadratic.
- **Kasai:**
  1. Compute LCP for suffix `banana$` (pos 0). Result = 0.
  2. Move to suffix `anana$` (pos 1). We know it shares a prefix with the predecessor of `banana$`.
  3. If prev LCP was H, we start comparing from $H - 1$. We skip characters we know must match.

2.2 The Inequality Proof

Let $rank[i]$ be the position of suffix i in the Suffix Array. Let $H = LCP[rank[i]]$. We want to prove:

$$LCP[rank[i+1]] \geq H - 1$$

Proof:

1. Let suffix j be the immediate predecessor of suffix i in the SA. ($rank[j] = rank[i] - 1$).
2. They share a prefix of length H.
$$suff_i = c \cdot \alpha \dots$$
$$suff_j = c \cdot \alpha \dots$$
3. Now consider $suff_{i+1}$ and $suff_{j+1}$. They are obtained by dropping the first char c.
$$suff_{i+1} = \alpha \dots$$
$$suff_{j+1} = \alpha \dots$$
They clearly share a prefix of length $H - 1$.
4. $suff_{j+1}$ appears somewhere before $suff_{i+1}$ in the SA (lexicographical order is usually preserved unless the first char differed).
5. The LCP between $suff_{i+1}$ and its *immediate* predecessor in SA must be $\geq$ the LCP between $suff_{i+1}$ and *any* predecessor (including $suff_{j+1}$).
6. Therefore, the stored LCP value for $i + 1$ is at least $H - 1$.

2.3 Complexity Analysis

- **Increments:** In the `while` loop, we compare characters to increase H. Since H starts at 0 and max is N, total increments $\leq 2N$.
- **Decrements:** In every step $i \to i + 1$, H decreases by at most 1. Total decrements $\leq N$.
- **Total:** Time is proportional to increments + decrements = $O(N)$.

---

3. Suffix Trees from SA + LCP

A Suffix Tree can be viewed as the **Cartesian Tree** of the LCP array.

Construction:

1. Insert suffixes in SA order.
2. Use $LCP[i]$ to determine how high up the rightmost path of the tree we must go to attach the new leaf.
   - If $LCP[i]$ matches the string depth of a node u, attach as child.
   - If $LCP[i]$ falls in the middle of an edge, **Split** the edge and attach.
3. All operations are amortized $O(1)$ (similar to the "Rightmost path" logic in Cartesian tree building).
4. **Total Time:** $O(N)$.

---

4. Text Mining with SA

Problem: Given pattern P, pattern Q, and distance k. Find if there exists an occurrence of P and Q in T such that distance is $\leq k$.

Algorithm:

1. **Search:** Use Binary Search on SA to find the range of suffixes for P ($[sp, ep]$) and Q ($[sq, eq]$).
2. **Collect:** We have two sets of positions: $S_P = \{SA[i] \mid sp \leq i \leq ep\}$ and $S_Q = \{SA[j] \mid sq \leq j \leq eq\}$.
3. **Merge/Plane Sweep:**
   - Sort S_P and S_Q (if not already sorted by position).
   - Iterate through the sorted lists. For each $p \in S_P$, check if any $q \in S_Q$ falls in $[p - k, p + k]$.
   - Since lists are sorted, this check is linear $O(|S_P| + |S_Q|)$.

Complexity: $O(N)$ (Build SA) + $O(|P| \log N + |Q| \log N)$ (Search) + $O(\text{occ} \log \text{occ})$ (Sort positions) + $O(\text{occ})$ (Sweep).

Hashing Protocols

This chapter covers advanced hashing techniques that provide probabilistic guarantees on performance, moving beyond simple heuristics.

---

1. Universal Hashing

Standard hashing uses a fixed function $h(x)$. This is vulnerable to adversarial attacks (a user can deliberately input keys that all map to the same slot, causing $O(N)$ performance). **Universal Hashing** solves this by selecting a hash function *randomly* from a carefully designed family at runtime.

1.1 The Definition

A family of functions $\mathcal{H} = \{h : U \to \{0, \ldots, m-1\}\}$ is **Universal** if, for any two distinct keys $x, y \in U$:

$$P_{h \in \mathcal{H}}(h(x) = h(y)) \leq \frac{1}{m}$$

- **Implication:** The collision probability is the same as if the hash function were truly random (uniform).
- **Chain Length:** If we use chaining with a universal hash function, the expected length of a chain is $1 + \alpha$ (where $\alpha = N/m$), guaranteeing $O(1)$ average access.

1.2 Example: The Class $\mathcal{H}_{a,b}$

We can construct a universal family using a large prime $p > |U|$. For any $a \in \{1, \ldots, p-1\}$ and $b \in \{0, \ldots, p-1\}$, define:

$$h_{a,b}(x) = ((ax + b) \mod p) \mod m$$

- **Randomization:** At program start, we pick random integers a and b.
- **Why it works:** The map $x \to (ax + b) \mod p$ is a bijection on the field $\mathbb{Z}_p$. It "scrambles" the input uniformly before mapping to m buckets.

---

2. The Power of Two Choices

In standard chaining, even with a perfect hash function, the maximum chain length (Max Load) grows with N.

- **One Choice:** If we throw N balls into N bins randomly:

$$\text{Max Load} \approx \frac{\ln N}{\ln \ln N}$$

- **Two Choices:** If for every ball, we compute **two** hashes $h_1(x), h_2(x)$, inspect the load of both buckets, and place the ball in the **less loaded** one:

$$\text{Max Load} \approx \frac{\ln \ln N}{\ln 2} + O(1)$$

- **Impact:** This is an exponential improvement. For $N = 10^6$, the max load drops from ≈ 9 to ≈ 3.

---

3. Cuckoo Hashing

Cuckoo Hashing leverages the "Power of Two Choices" to achieve $O(1)$ **Worst-Case Lookup Time**.

3.1 The Algorithm

- **Structure:** Two tables T_1, T_2 of size m. Two universal hash functions h_1, h_2.
- **Invariant:** A key x is stored **either** at $T_1[h_1(x)]$ **or** at $T_2[h_2(x)]$.

Lookup(x)

1. Check $T_1[h_1(x)]$. If found, return.
2. Check $T_2[h_2(x)]$. If found, return.
3. Else, not found.

- **Cost:** Exactly 2 memory accesses. **Deterministic** $O(1)$.

Insert(x)

1. Try to place x in $T_1[h_1(x)]$.
2. If empty, done.
3. If occupied by key y, **kick out** y and replace with x.
4. Now insert y into its *alternative* location (e.g., if y was in T_1, try $T_2[h_2(y)]$).
5. If that spot is occupied by z, kick out z and repeat.
6. **Rehash:** If this process loops too long ($> MaxLoop$), assume a cycle and rebuild tables with new hash functions.

3.2 The Cuckoo Graph

We can model the state as a graph:

- **Nodes:** The $2m$ slots in the tables.
- **Edges:** For every key k, an edge connects $h_1(k) \leftrightarrow h_2(k)$.
- **Insertion:** Insertion succeeds if the connected component containing the new edge is a **Tree** or has at most **One Cycle** (Unicyclic).
  - We can orient edges to point to where the key *is currently stored*.
  - A tree or 1-cycle component can always be oriented such that every key has a home.
- **Failure:** If the component contains **two cycles** (a "Bicycle" or "Figure-8"), insertion is impossible. The keys will kick each other around infinitely.

3.3 Analysis

- **Load Factor:** Cuckoo hashing works if $\alpha < 0.5$ (tables less than half full).
- **Rehash Probability:** With $\alpha < 0.5$, the probability of forming a bicycle is $O(1/N)$.
- **Amortized Cost:** Since rehashing is rare, expected insertion time is **constant** $O(1)$.

Filters and Perfect Hashing

This chapter focuses on space-efficient structures for set membership and static dictionary problems.

---

1. Minimal Ordered Perfect Hashing (MOPHF)

1.1 The Goal

Given a **static** set of keys S ($|S| = N$), we want to construct a hash function $h(x)$ such that:

1. **Perfect:** No collisions for $x \in S$.
2. **Minimal:** The range of $h(x)$ is exactly $\{1, \ldots, N\}$.
3. **Ordered:** If $x < y$, then $h(x) < h(y)$. This means $h(x)$ returns the **rank** of x in S.

1.2 Construction (3-Hypergraph / Bucketing)

We use a 2-level construction involving random hypergraphs.

- **Hash Functions:** We pick two universal hash functions h_1, h_2 mapping to a larger range $M' \approx cN$.
- **The Graph:** Construct a graph where nodes are $0 \ldots M' - 1$ and each key $k \in S$ is an edge between $h_1(k)$ and $h_2(k)$.
- **Acyclicity:** We retry choosing h_1, h_2 until this graph is **acyclic**.
- **Labeling (g):** We assign values to nodes (g array) such that for every edge $k = (u, v)$, the rank of k is derived from $g[u] + g[v]$.
$$h(k) = (g(h_1(k)) + g(h_2(k))) \mod N$$
- **Efficiency:**
  - **Time:** $O(N)$ expected construction, $O(1)$ worst-case query.
  - **Space:** $O(N)$ bits (storing the g array).

---

2. Bloom Filters

A probabilistic data structure that tests membership.

- **False Negatives:** Impossible. (If it says "No", it's definitely not there).
- **False Positives:** Possible with probability f. (If it says "Yes", it might be there).

2.1 Algorithm

- **Storage:** An array B of M bits, initialized to 0.
- **Hashes:** k independent hash functions $h_1, \ldots, h_k$.

Insert(x): Set $B[h_i(x)] = 1$ for all $i = 1 \ldots k$.

Query(x): Return **True** if $B[h_i(x)] == 1$ for all $i = 1 \ldots k$. Else return **False**.

2.2 Error Analysis

Let N be the number of inserted items.

1. **Prob(Bit is 0):** The probability that a specific bit is *not* set by one specific hash function during one insertion is $1 - 1/M$. After N insertions (k hashes each):

$$p_0 = \left(1 - \frac{1}{M}\right)^{kN} \approx e^{-kN/M}$$

2. **False Positive Rate (f):** A query for a non-existent element returns true if all k corresponding bits happen to be 1.

$$f = (1 - p_0)^k \approx \left(1 - e^{-kN/M}\right)^k$$

2.3 Optimal k

To minimize f, we differentiate with respect to k. The optimal number of hash functions is:

$$k = \frac{M}{N} \ln 2 \approx 0.7 \frac{M}{N}$$

Substituting this back into the error formula, the optimal error rate is:

$$f_{opt} = \left(\frac{1}{2}\right)^k = (0.6185)^{M/N}$$

2.4 Application: Approximate Set Intersection

Scenario: Two servers A and B want to compute $|A \cap B|$ (e.g., Netflix CDNs syncing files) without sending the full dataset. **Protocol:**

1. A sends $BF(A)$ to B.
2. B checks every item $y \in B$ against $BF(A)$.
3. B counts the matches. **Error:** The result is $|A \cap B|$ + False Positives.

- Expected False Positives $\approx |B \setminus A| \times f$.
- To improve accuracy, B can send the "suspected" intersection back to A for exact verification (2-round protocol).

---

3. Spectral Bloom Filters (SBF)

Standard Bloom Filters cannot handle **deletions** or **frequency estimation**.

3.1 Structure

Replace the bit array B with an array of **counters** C.

- **Insert(x):** Increment $C[h_i(x)]$ for all i.
- **Delete(x):** Decrement $C[h_i(x)]$.
- **Query Frequency(x):**
$$\text{freq}(x) \approx \min_i \{C[h_i(x)]\}$$
  - *Why Min?* Collisions only *add* to the counter. The counter with the least noise is the closest upper bound to the true frequency.

Data Compression

We focus on **lossless** statistical compression. The goal is to represent a message S of length n using the minimum number of bits possible, bounded by the entropy of the source.

1.1 Compression Models

1. **Static:** The frequency model is fixed (e.g., standard English letter frequencies). Fast, but poor compression if data deviates from standard.
2. **Semi-Dynamic:** Two-pass. Pass 1 counts frequencies and builds the model (stored in header). Pass 2 encodes data. Good compression, slower.
3. **Dynamic:** One-pass. The model is updated on the fly as symbols are read. Decoder mirrors the updates.

---

2. Information Theory Basics

2.1 Entropy (H_0)

Entropy is a measure of the information content or "randomness" of a source. For a source alphabet Σ where each symbol σ appears with probability $P(\sigma)$, the **0-th order Entropy** is:

$$H_0 = \sum_{\sigma \in \Sigma} P(\sigma) \log_2 \left(\frac{1}{P(\sigma)} \right) = -\sum_{\sigma \in \Sigma} P(\sigma) \log_2 P(\sigma)$$

- **Meaning:** H_0 is the theoretical lower bound on the average number of bits per symbol needed to represent the source. No compressor can beat H_0 (on average) if it treats symbols independently.

2.2 Kraft's Inequality

For any instantaneous (prefix-free) code with codeword lengths $\ell_1, \ell_2, \ldots, \ell_{|\Sigma|}$, the lengths must satisfy:

$$\sum_{i=1}^{|\Sigma|} 2^{-\ell_i} \leq 1$$

- **Why?** This condition ensures that the codewords can be mapped to the leaves of a binary tree, which guarantees unique decodability without lookahead.

---

3. Huffman Coding

Huffman coding is a greedy algorithm that produces an optimal prefix-free code for a given distribution.

3.1 Construction (Min-Heap)

1. **Init:** Create a leaf node for each symbol σ with weight $P(\sigma)$. Insert all into a Min-Heap.
2. **Loop:** While Heap size > 1:
   - Extract two nodes with smallest weights: u, v.
   - Create a new internal node *parent* with weight $w(u) + w(v)$.
   - Make u and v children of *parent*.
   - Insert *parent* back into Heap.
3. **Result:** The code for symbol σ is the path from Root to Leaf σ (0 for left, 1 for right).

3.2 Canonical Huffman

Standard Huffman trees are hard to store (need pointers). **Canonical Huffman** allows us to reconstruct the code using **only** the lengths of the codewords.

Reconstruction Algorithm:

1. **Input:** Array `num[L]` (count of symbols with length L) and `symb[L]` (list of symbols of length L, sorted lexicographically).
2. **Compute `fc` (First Codeword):** We calculate the integer value of the first codeword for each length L.
   - Start from max length: $fc[max_len] = 0$.
   - Iterate backwards from $i = max - 1$ down to 1:

$$fc[i] = \lfloor \frac{fc[i+1] + num[i+1]}{2} \rfloor$$

   - *Logic:* This formula effectively performs a right shift. We take the "next available" integer at level $i + 1$ and shift it right to find the prefix at level i.
3. **Assign:** For length L, the k-th symbol in `symb[L]` gets code $fc[L] + k$.

3.3 Extended Huffman

Standard Huffman has an overhead of +1 bit per symbol in the worst case ($H \leq L < H + 1$).

- **Idea:** Group symbols into blocks of size k (k-tuples).
- **Alphabet:** New alphabet size $|\Sigma|^k$.
- **Overhead:** The +1 bit overhead is distributed over k symbols.

$$L_{avg} \approx H + \frac{1}{k}$$

- **Trade-off:** The tree size grows exponentially with k.

---

4. Arithmetic Coding

Huffman has a limitation: it must use an integer number of bits (at least 1) per symbol. **Arithmetic Coding** overcomes this by mapping the entire message to a single number (interval).

4.1 The Interval Concept

We represent the current state as an interval $[L, L + S)$ within $[0, 1)$.

- Initially: $[0, 1)$.
- Each symbol σ owns a sub-interval proportional to $P(\sigma)$.
- When we process σ, we "zoom in" to its sub-interval.

4.2 Encoding Algorithm

Let $f(\sigma)$ be the cumulative probability (sum of $P(x)$ for all $x < \sigma$). For each symbol c in message:

1. New Size: $S_{new} = S_{old} \times P(c)$
2. New Start: $L_{new} = L_{old} + S_{old} \times f(c)$
3. Update $S \leftarrow S_{new}, L \leftarrow L_{new}$.

Result: A number $V \in [L_{final}, L_{final} + S_{final})$.

4.3 Comparison: Huffman vs. Arithmetic

Consider a symbol with $P(A) = 0.99$.

- **Entropy:** $H(A) = \log_2(1/0.99) \approx 0.014$ bits.
- **Huffman:** Must assign at least 1 bit to 'A'. Efficiency is terrible (1 vs 0.014).
- **Arithmetic:** The interval shrinks by factor 0.99. After 100 'A's, size is $0.99^{100} \approx 0.36$. We still barely need 1-2 bits to encode 100 symbols.
- **Theorem:** Arithmetic coding uses at most $nH_0 + 2$ bits total. The overhead is negligible for large n.